CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology and Engineering — CSPIT-IT

ITUE301 — Advanced Web Development Frameworks

B.Tech. – CSE, CE, IT | Semester: 5th | AY 2026–27

OPEN-BOOK PRACTICAL EXAMINATION

SET B — Library Book Management System

Duration	2 Hours
Total Marks	20
Tech Stack	React + Express.js + MongoDB (Mongoose)
Examination Type	Open-Book Practical Examination
Syllabus Coverage	Practical 1 to Practical 5
Internet	Permitted for documentation/reference
AI Tools	Permitted
Code Sharing	Strictly prohibited

General Instructions

1. Read the complete question before starting implementation.
2. Attempt all five tasks. Partial implementation will receive partial marks.
3. Use:
   - React for frontend tasks
   - Express.js for backend tasks
   - MongoDB with Mongoose for database tasks
4. Your GitHub repository must be public and named:

```
itue301-exam-[your-roll-number]-[batch]
```

Example: itue301-exam-24cse001-A

5. Use .env for the MongoDB connection string. Do not hardcode the connection string.
6. Commit an .env.example file. Do not commit .env.
7. The backend must start using:

```
node server.js   (or)   npm start
```

8. Internet access is permitted for reading documentation and references.
9. AI tools such as ChatGPT, GitHub Copilot, Claude and Gemini are permitted. Students must be able to explain the submitted code during viva.

10. Sharing code with another student is strictly prohibited.
11. Submit the GitHub repository link along with SHA (commit ID) before the examination ends.
12. Include a README.md containing basic instructions for running the frontend and backend.

Scenario: Library Book Management System

The college library wants to digitize basic information about its books, members and borrowing records.

You are required to develop selected parts of a Library Book Management System.

Data Entities

Use the following data structures.

Book

- title
- author
- category
- isbn
- available

Member

- name
- email
- phone
- department

Borrowing

- memberId
- bookId
- borrowDate
- returnDate
- status

The borrowing status must be one of:

```
borrowed   returned   overdue
```

Task 1 — React Component Architecture

4 Marks

Create the basic React frontend for the Library Book Management System.

Create the following components/pages:

- HomePage

- BooksPage
- BorrowPage
- BookCard

Place reusable components inside a suitable /components folder.

The BookCard component must accept the following props:

```
title
author
category
available
```

Display all four values.

The availability status must have a different visual appearance depending on whether the book is available or unavailable. For example:

- available = true → Available
- available = false → Not Available

Use props to pass book information from the parent component to BookCard.

Focus on component structure, composition and props. Detailed styling is not required.

Task 2 — React Routing and State Management

4 Marks

Configure React Router with the following routes:

Route	Component
/	HomePage
/books	BooksPage
/borrow	BorrowPage

Create a navigation component containing links to all three routes.

Navigation must be performed using React Router links without a full-page reload.

In BorrowPage, create a simple borrowing form containing at least:

- Member name
- Book title
- Borrow date
- Return date

Use useState to manage the form data.

At least two state values must be used meaningfully. For example:

- selected book
- member name
- borrow date

- return date

Display at least one entered/selected value on the page as the state changes.

The form input should behave as a controlled component using value and onChange.

Task 3 — Express REST API + Middleware

4 Marks

Create an Express backend for managing borrowing records.

Implement the following three REST endpoints:

Method	Endpoint	Purpose
GET	/api/v1/borrowings	Return all borrowing records
POST	/api/v1/borrowings	Create a new borrowing record
GET	/api/v1/books	Return all books

For this task, borrowing records and books may initially be stored in in-memory arrays (e.g., a small hardcoded books list on the server). MongoDB implementation is assessed separately in Task 5.

Create a custom requestLogger middleware.

For every request, it should log:

```
[METHOD] [PATH] [TIMESTAMP]
```

Example:

```
[GET] /api/v1/borrowings [2026-08-20T10:15:20.000Z]
```

Apply the middleware globally.

Also implement a global error-handling middleware as the last middleware in the application.

It should return a structured JSON response instead of exposing the raw error stack.

Use appropriate HTTP status codes. In particular:

- 200 for successful GET
- 201 for successful POST
- 500 for an unhandled server error

Test the APIs using Postman or Thunder Client.

Task 4 — REST API Consumption in React

4 Marks

Use the Express API developed in Task 3 from the React frontend.

In BooksPage, retrieve book information using:

```
GET /api/v1/books
```

Use fetch or Axios.

Use useEffect() so that the API request is made when the component is mounted.

Maintain three states:

```
data, loading and error
```

The page must:

1. Display a loading message/indicator while the request is in progress.
2. Display an error message if the request fails.
3. Display the book data after a successful request.
4. Display at least:
   - Book title
   - Author
   - Availability

The book information must be rendered from the API response and must not be hardcoded in the React component.

The API call should follow the asynchronous pattern.

Task 4 consumes the Express API from Task 3. MongoDB is not involved in this flow.

Task 5 — MongoDB + Mongoose Schema Design and Validation

4 Marks

Create a separate Mongoose-based implementation for the library database. Connect to MongoDB using Mongoose.

Create the following three schemas.

Book Schema

Field	Requirement
title	String, required
author	String, required
category	String, required
isbn	String, unique
available	Boolean, default true

Member Schema

Field	Requirement
name	String, required
email	String, required, unique
phone	String
department	String, required

Borrowing Schema

Field	Requirement
memberId	Reference to Member
bookId	Reference to Book
borrowDate	Required
returnDate	Required
status	Enum

The status field must use:

```
borrowed   returned   overdue
```

with borrowed as the default value.

Use Mongoose references for:

```
memberId → Member
bookId → Book
```

Connect MongoDB using a connection string stored in .env. For example:

```
MONGO_URI=your_mongodb_connection_string
```

Create at least one MongoDB operation that demonstrates that the schema is working.

Demonstrate at least one validation failure, such as:

- missing required field
- missing member name
- missing book title
- invalid borrowing status

Return a meaningful JSON error response instead of exposing the raw Mongoose error object.

Submission Requirements

The GitHub repository must contain:

```
itue301-exam-[roll-number]-[batch]/
│
├── frontend/
│   ├── src/
│   └── package.json
│
├── backend/
│   ├── models/
│   ├── server.js
│   └── package.json
│
├── .env.example
├── .gitignore
└── README.md
```

The README must briefly explain:

1. Project name

2. Frontend setup and run command
3. Backend setup and run command
4. MongoDB setup
5. Required environment variables

Report / Evidence

Submit a short PDF report containing three screenshots.

Screenshot 1 — React Application

Show the Library Book Management System running in the browser.

Screenshot 2 — REST API

Show Postman/Thunder Client displaying a successful request to one of the Express endpoints.

Screenshot 3 — MongoDB

Show MongoDB Compass or MongoDB Atlas displaying a created document.

PDF filename:

```
[RollNo]_SetB_Report.pdf
```